D1 Detailed Design: Calorie-Based Meal Finder

Assignment 6 | Team members: Eric Braverman, Brayden Molinyawe, Christopher Farah

1. Header, scope, and conventions

Project title: Calorie-Based Meal Finder

Goal statement: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

D1 details **C2 Food Search API, C4 Nutrition Data Importer, and C5 PostgreSQL Database**; D2 finishes C1 Web Frontend, C3 User Profile API, and the shared integration behavior. These are modules of the single Spring Boot backend already chosen in D0, not new services. The Next.js frontend, Keycloak, USDA FoodData Central, Docker Compose, Debian VM, and Cloudflare Tunnel remain the D0 architecture.

The input is a logged-in user's calorie target, optional restaurant filter, and maintained nutrition data. The output is a ranked list of meals or grocery items at or below the target, with calories, portions, and item details. Calorie-only stories US-01 through US-05 define this release. US-06 through US-08 describe a proposed macro extension; protein ranking, saved macro targets, preferences, and guest search remain deferred because the supplied project notes leave decisions open.

Diagram conventions: Every figure repeats the project title and goal. Blue rounded boxes are steps or modules the team builds; green boxes are persistent data; gray dashed boxes are configured external systems; yellow boxes are states or user-visible outcomes. Solid arrows show flow unless a data-model figure says they are relationships; red dashed arrows show errors. PK means primary key, FK means foreign key, and cardinality labels show the allowed count at each relationship end. C1-C5, E1-E3, I1-I13, US and AC IDs retain their D0 meanings.

Design assumptions: Plan for 10,000 meals, 50,000 food items, 200,000 meal-component rows, and up to 50 simultaneous users. These are sizing assumptions, not measurements or new acceptance criteria. The 100-times case is 1 million meals, 5 million items, and 20 million component rows. Only the existing two-second search requirement is a committed response-time requirement. New limits below are explicit implementation decisions for team review.

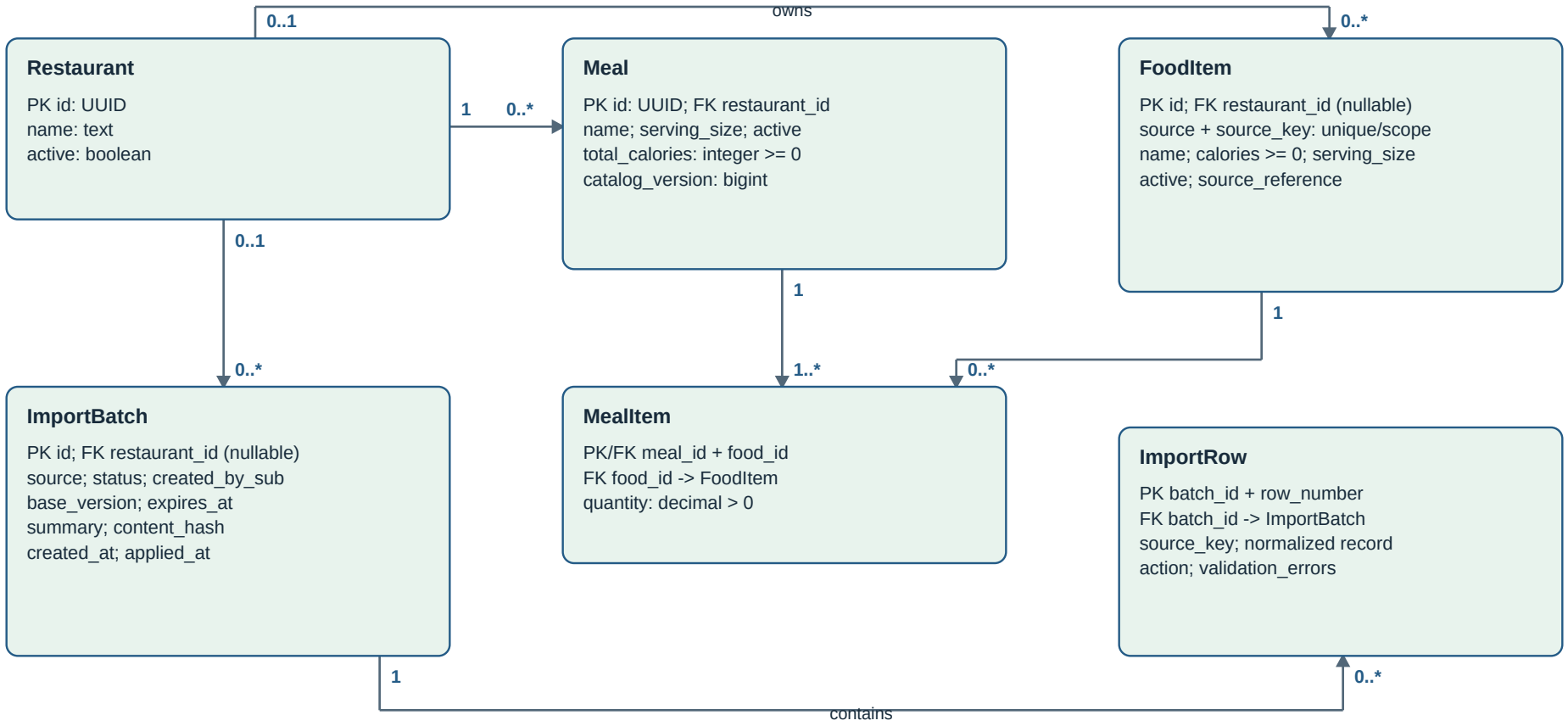
Ownership: Eric: C2/C4; Christopher: C3/C5 and infrastructure; Brayden: C1. These follow the provisional D0 ownership table; everyone still works across the stack.

2. Data model, the D1 diagram

Calorie-Based Meal Finder | D1 data model (C5)

Goal: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

Green boxes = persistent entities. PK/FK identify keys. Edge labels show cardinality at both ends; lines are relationships.



CatalogState singleton (id=1, version: bigint) serializes confirmed imports; unrelated to any user account.

A valid active Meal has >= 1 component. Invalid staged imports may have 0 normalized rows. No passwords in C5.

Structural decisions and integrity rules

Decision	Structure and reason	Index / enforcement
Separate FoodItem and Meal	A food serving can appear in several complete meals. A meal is a named, curated combo, not an arbitrary calorie combination assembled at search time. Grocery search returns one FoodItem per result.	UUID primary keys. FoodItem has nonblank name, serving_size, source, and source_reference; nonnegative integer kcal.
Model composition explicitly	MealItem resolves the Meal-to-FoodItem many-to-many relationship and stores the quantity of the item's stated serving used in that meal.	Composite PK (meal_id, food_id); quantity numeric(10,3) > 0; FK restrictions preserve references. Active meals must have at least one component.
Restaurant cardinalities	Each Meal belongs to exactly one Restaurant. A restaurant item belongs to one Restaurant; a USDA grocery item has no restaurant. Components of a restaurant meal must belong to that restaurant.	FK restaurant_id plus source-specific validation in the write service. Restaurant-to-Meal is 1 to 0..many; Restaurant-to-FoodItem is 0..1 to 0..many.
Precompute meal totals	Recalculate from quantities when an import changes ingredients. Searches do not build combinations or sum the whole catalog.	total_calories = round-half-up(sum(food calories * quantity)), rounded once per meal. Use the same rule for rendered component contributions, with an explicit rounding adjustment when required.
Stable upstream identity	Names can change. Match restaurant rows by restaurant_id + source + source_key; USDA rows use source + FDC id.	Partial unique indexes for restaurant-scoped and grocery-scoped source keys. No name-based deduplication.
Keep import staging separate	ImportBatch holds status, summary, source, baseline version, and expiry. ImportRow stores normalized proposed rows or row errors; this permits confirmation after refresh or restart.	ImportRow PK (batch_id, row_number), FK batch_id. Index batches on (status, expires_at). Delete abandoned staging after seven days; catalog/source provenance remains on active food records.
Prevent stale confirmation	CatalogState is one persistent row with an increasing version. Preview records the current version; confirm locks and compares it.	Row lock serializes writes; a changed version returns 409 and requires a fresh preview. All C4 writes use this gate.
Removal without broken meals	A missing row in a restaurant full snapshot is a removal. Mark the item inactive; any meal containing it becomes inactive. Preserve its value and components for integrity.	No hard deletion of referenced food. Recompute affected totals, even when the resulting meal is hidden. A later reactivation can restore eligibility.

Use a relational store because composition, scoped uniqueness, and atomic multi-table updates are part of correctness. A JSON-only store would move those checks into application code. C3's UserProfile and deletion coordination entities are completed in D2; no identity credentials are stored in this nutrition model.

Search indexes and read consistency

Use a partial B-tree index on Meal `(total_calories DESC, id ASC) WHERE active = true`, a second on `(restaurant_id, total_calories DESC, id ASC) WHERE active = true`, and an equivalent calories/id partial index on active grocery FoodItem rows. The MealItem primary key supports component lookup by meal_id; add `(food_id, meal_id)` to find meals affected by an import. A restaurant filter containing several IDs can require merging or sorting qualifying rows; the design does not assume one index always removes that work.

The I10 read runs in a short, read-only **REPEATABLE READ** transaction: choose the top IDs, fetch their components, and, if needed, choose the fallback within the same snapshot. A concurrent import therefore cannot produce a meal total from one version and components from another. C2 uses a read-only database role; only C4 can change catalog tables. A separate C3 role writes profile/deletion tables and can call the attribution-erasure function described in D2, without nutrition-write privileges. Flyway-style ordered SQL migrations are the proposed schema mechanism; migrations are not generated by the running application.

At meal search time, eligibility requires an active meal, an active restaurant, and active components; the import transaction maintains the meal's active flag. Grocery eligibility requires an active USDA/grocery row. An unknown restaurant UUID is syntactically valid but yields no eligible records. A restaurant filter on a GROCERY request is rejected as contradictory input rather than silently ignored.

Important boundary: Item calories are per the exact serving_size shown, not per an assumed 100 g. A MealItem quantity of 2 means two of that item's stated servings. Restaurant CSV values are already per serving. USDA energy values are normalized only when their basis and units are known; if energy is per 100 g, the importer requires a convertible serving mass and calculates kcal per stated serving. A missing mass or an ambiguous basis is a row error. Energy may be zero, but may not be missing or negative.

Source metadata consists of a public source URL or USDA FDC identifier plus retrieval/import time. It must never contain the API key. Calorie values and serving sizes are information for comparison; the UI uses the required label **calorie match** and does not infer that a result is healthy.

3. Core algorithms

A1. Filter and rank calorie matches (C2)

What it is / problem solved: Return already-defined complete meals or single grocery items at or below an integer calorie limit, closest to that limit first. This implements US-01, US-02, and UC-01 without asking users to add ingredient calories or relying on an upstream API during a search.

Inputs / outputs with exact types: Input SearchQuery = {type: enum MEAL/GROCERY, targetCalories: int 100..5000, restaurantIds: UUID[] optional, limit: int 1..20, default 20}; authentication is a verified JWT subject. Output SearchResponse = {results: SearchResult[], closestOption: SearchResult or null}. SearchResult and errors are defined in Section 5. Each result contains named item entries with integer calories, serving-size strings, and a positive decimal quantity.

Expected complexity at realistic size: A single unfiltered indexed top-k read is approximately O(log N + k + J), where k <= 20 and J is the returned component count. Loading all components uses one batched query, not one query per meal. A multi-restaurant plan can instead scan M qualifying entries and sort in O(M log M); inspect its plan rather than claiming the best-case bound universally. At N = 10,000 and 100N = 1,000,000 meals, balanced-tree depth grows logarithmically; broad filters, disk reads, and concurrency still require measurement. The database part must stay within D0's 500 ms allocation; the full validate/query/rank stage has 800 ms. No benchmark has been run on an implementation.

Why this rather than alternatives: For a fixed upper calorie bound, smallest target-minus-calories is equivalent to calories descending. Ranking in indexed SQL avoids loading/sorting the entire catalog in Java. A knapsack algorithm would generate unapproved combinations, while the requirement asks for stored complete meals. Equal-calorie results use id ascending for deterministic output.

Edge cases: Reject malformed or non-whole input before SQL; bind all parameters. If there are no matches, return an empty results array and the lowest-calorie active option within the selected scope, even if it exceeds the target. If the scope has no options at all, closestOption is null. Return 503 on database failure with no partial results. Use the same snapshot for results, fallback, and details.

```
validate type, target, limit, restaurantIds
begin read-only REPEATABLE READ
  ids = eligible rows WHERE calories <= target
        ORDER BY calories DESC, id ASC LIMIT limit
  results = batch-load typed rows and components for ids
  closest = null
  if ids is empty:
    closest = lowest eligible row ORDER BY calories ASC, id ASC LIMIT 1
    load its components in the same snapshot
commit read transaction
return {results, closestOption: closest}
```

A2. Validate, preview, and atomically apply nutrition changes (C4)

What it is / problem solved: Validate every incoming nutrition row, show an exact add/remove/change preview, and apply only the confirmed, unchanged preview. US-03 requires invalid imports to leave the catalog untouched and requires every affected meal total to be recalculated.

Inputs / outputs with exact types: Input is a UTF-8 CSV <= 5 MiB for one existing restaurant, or a USDA query string producing an explicitly selected grocery set. CSV fields: itemId: string 1..100, name: string 1..200, calories: int >= 0, servingSize: nonblank string <= 200, sourceUrl: HTTPS URL. Restaurant CSV is a complete snapshot; USDA selection is upsert-only and never deletes other grocery rows. Output ImportResponse is typed in Section 5: batch UUID, status enum, added/removed counts, changed entries, and mealsRecalculated count. Invalid data returns row numbers and reasons, with no live nutrition writes.

Expected complexity at realistic size: For R staged rows, E current items in scope, A affected MealItem links, and V affected meal components, hash comparison and recomputation are O(R + E + A + V) time and O(R + E) memory. Bound CSV to 20,000 data rows and text field lengths, and bound a USDA selection to 1,000 records. At 100 times the catalog size, a fixed restaurant batch remains bounded; scanning all 5 million items would be inappropriate. Scope comparison queries and the food_id index avoid that global scan. Large global recalculations would need streaming/staging joins; they are outside this initial import limit.

Why this rather than alternatives: Stable source keys avoid false matches after renames. A preview avoids silent changes, and one database transaction avoids a half-updated catalog. Recalculating only affected meals avoids a full catalog pass. CSV parsing, HTTP calls, and database transactions are reused libraries; only project-specific validation, diffing, and calorie aggregation are built.

Edge cases / failure behavior: Reject the whole input on an empty CSV, duplicate source key, missing serving size, invalid units, negative/decimal kcal, or unsupported source mapping. CSV header is line 1; row errors use physical line numbers, including blank lines. Malformed quoting rejects the file with its parser line number. For USDA, report record index plus FDC id when available. A no-change preview ends immediately with zero counts and NO_CHANGES. Confirming an expired preview gives 410; a changed catalog gives 409. Lock CatalogState and the ImportBatch during confirmation. Applying an already APPLIED batch returns its saved result, with no duplicate mutations.

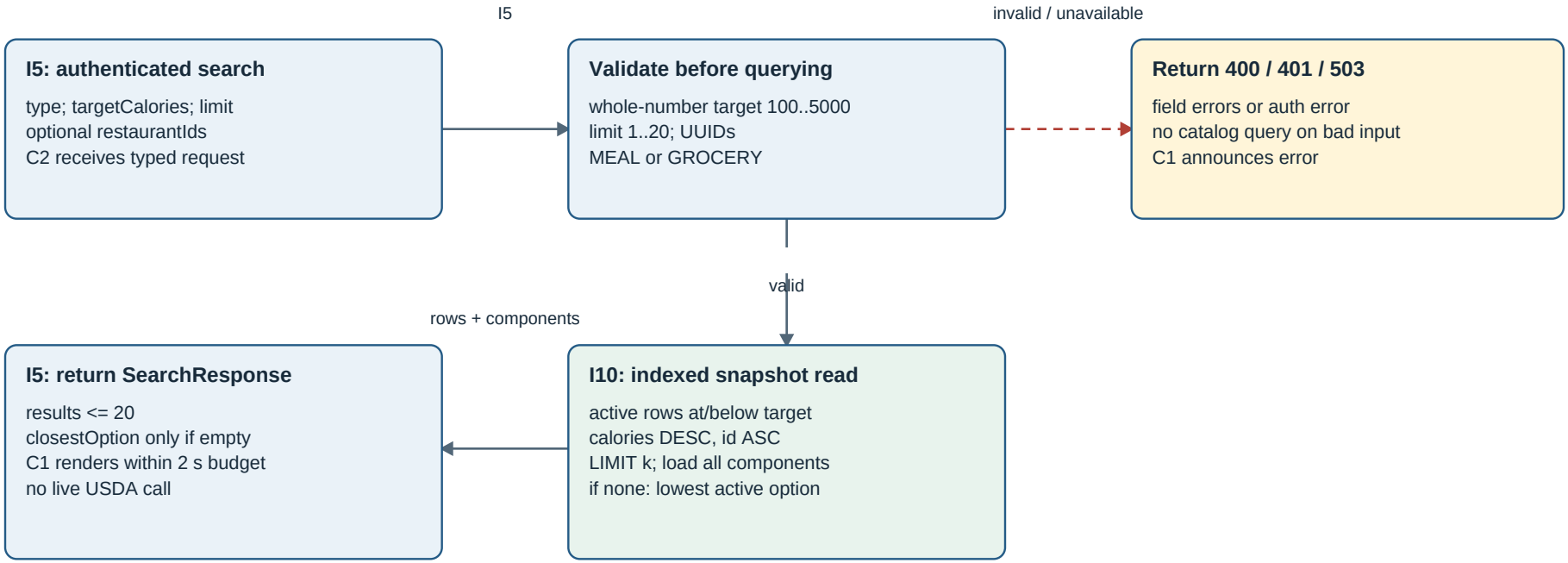
```
preview: authorize maintainer; enforce limits; parse + normalize all rows
         if any error: save REJECTED metadata; return 422 with row errors
         diff stable keys against current scope + record catalog version
         if diff empty: return NO_CHANGES
         persist PENDING_CONFIRMATION batch/rows; return exact preview
confirm: authorize; lock batch; if APPLIED return saved response
         check owner/role, expiry and catalog version under CatalogState lock
         in one transaction: apply changes, recalculate affected meals,
         increment version, set APPLIED + summary, commit
         on known rollback: separately mark batch FAILED
         on uncertain commit: reload batch status before any retry
```

D1 behavior diagram: search

Calorie-Based Meal Finder | D1 search algorithm (C2)

Goal: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

Blue boxes = steps. Solid arrows = request/data flow. Red dashed arrows = errors. I5/I10 retain D0 interface IDs.



Tie rule: UUID ascending for equal calories. Restaurant filter applies to normal and fallback results.

0 matching rows is HTTP 200. A fallback above the target is explicitly labeled, never presented as a match.

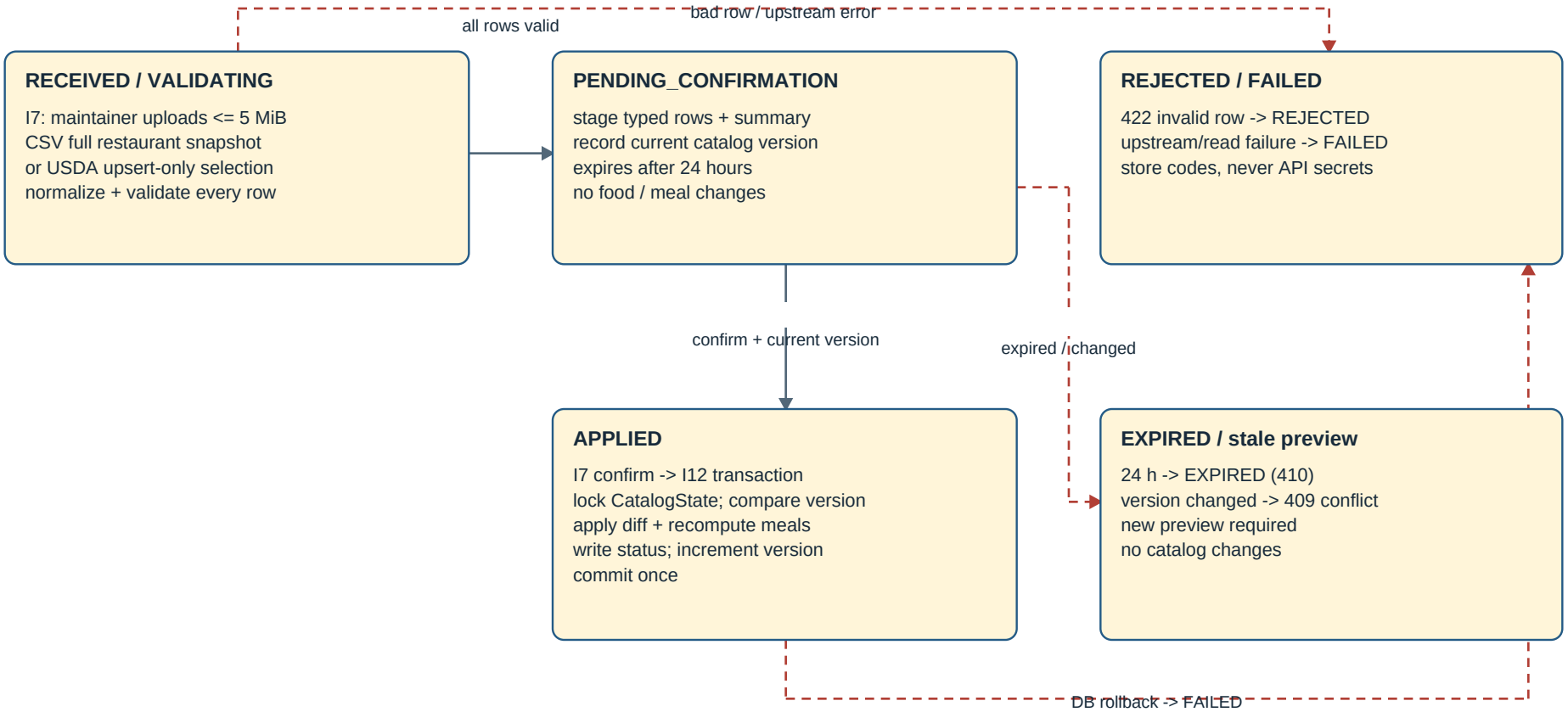
Database failure -> 503; no partial list. Read-only REPEATABLE READ prevents mixed catalog snapshots.

D1 behavior diagram: import and confirmation

Calorie-Based Meal Finder | D1 importer lifecycle (C4)

Goal: Help people find fast-food meals and grocery items that fit a personal calorie target, with clear calorie counts and serving sizes.

Yellow boxes = persisted batch states. Solid arrows = transitions; red dashed arrows = failure paths. I7/I12 match D0.



Repeated confirmation of APPLIED returns its saved summary (200), with no second mutation or version increment.

No-change input: return NO_CHANGES and zero counts; no staged confirmation and no catalog version increment.

4. Build-versus-reuse decisions

License names below refer to the selected upstream projects, not to a legal review of every transitive dependency. Pin supported releases and record the dependency inventory before implementation. Official references are listed in Section 6.

Significant piece	Build or reuse / candidate	License and maturity	Performance and fit / reason
Calorie search and fallback	Build domain query and response mapping on Spring JDBC	Team-owned logic; JDBC/Spring are established database access mechanisms.	Indexed top-k query implements the exact cutoff and fallback rules. A generic recommendation library does not know these requirements.
Meal totals and import diff	Build stable-key comparison and affected-meal recalculation	Team-owned business logic, with review against AC-03.1 through AC-03.3.	Batch comparison and one transaction fit the expected data size. Do not hand-write parsing or transaction infrastructure.
Backend HTTP and validation	Reuse Spring Boot, Spring MVC, Bean Validation	Spring Boot: Apache-2.0; mature Spring ecosystem. Verify selected validation implementation's license in the dependency inventory.	Typed controllers and validation fit team Java skills and the single backend image; avoid custom HTTP routing.
SQL storage and access	Reuse PostgreSQL and Spring JDBC; use PostgreSQL JDBC driver	PostgreSQL License; Spring Apache-2.0; driver BSD-2-Clause. Established relational tools.	Transactions and indexed joins support import correctness. JDBC exposes the search SQL and avoids opaque ORM query plans.
CSV and JSON parsing	Reuse Apache Commons CSV and Jackson	Commons CSV: Apache-2.0; Jackson core/databind: Apache-2.0. Established parsers; pin compatible versions.	Handle escaped fields and JSON typing with libraries; enforce file, row, and string limits before building an unbounded in-memory import.
JWT authentication	Reuse Spring Security OAuth2 resource server and Keycloak	Apache-2.0 projects with established OIDC support.	Standard signature/claim validation replaces custom token parsing; domain authorization still belongs in this project.
Frontend and session integration	Reuse Next.js/React and an established OIDC client such as openid-client	Next.js/React MIT; openid-client MIT. Confirm the chosen runtime-compatible release.	Server-side session handling fits D0's token placement. Build forms and result rendering; reuse the protocol implementation.
Deployment and HTTPS routing	Reuse Docker Engine/Compose, Debian, and cloudflared	Engine/Compose/cloudflared: Apache-2.0 projects; Debian is a collection with package-specific licenses. Cloudflare service terms are separate.	One host matches the class-project load. Compare a managed host or Nginx/Caddy before changing D0; no Kubernetes need is established.

Release check: Evaluate security maintenance and supported runtime compatibility when pinning versions. Check parser memory behavior with the 5 MiB limit, query plans with the sizing fixture, and session/JWT integration end to end. These are implementation validation tasks, not completed test results.

5. API contract

Shared wire types and rules

Paths are versioned under `/api/v1`. Browser requests go to same-origin C1; C1 forwards I5/I7 internally with `Authorization: Bearer <access-token>`. C2/C4 independently validate issuer, audience, signature, expiry, deletion state, and required role. Times use ISO 8601 UTC; IDs use canonical UUID strings; calories are integer kcal, counts are integers, quantities are decimal numbers in stated servings. No field means grams unless explicitly named as such.

```
{
  "results": [{
    "id": "11111111-1111-4111-8111-111111111111",
    "name": "Grilled Chicken Sandwich Meal", "resultType": "MEAL",
    "restaurant": "Example Burger Co.", "totalCalories": 680,
    "servingSize": "1 meal",
    "items": [
      {"name": "Grilled Chicken Sandwich", "calories": 390,
       "servingSize": "1 sandwich (204 g)", "quantity": 1},
      {"name": "Side Salad", "calories": 290,
       "servingSize": "1 bowl (150 g)", "quantity": 1},
      {"name": "Diet Cola", "calories": 0,
       "servingSize": "16 fl oz", "quantity": 1}
    ], "roundingAdjustmentCalories": 0
  }], "closestOption": null
}
```

SearchResult: id: UUID; name: string; resultType: MEAL/GROCERY; restaurant: string or null (null for grocery); totalCalories: int >= 0; servingSize: string; items: ItemResult[] with at least one item; roundingAdjustmentCalories: int. ItemResult = {name: string, calories: int >= 0 per stated serving, servingSize: string, quantity: decimal > 0}. The displayed total equals the sum of rounded component contributions plus the explicitly shown rounding adjustment. Grocery uses one item, quantity 1, adjustment 0. `closestOption` has the identical shape or is null.

ErrorResponse for every non-2xx application response: `{code: string, message: string, fieldErrors: FieldError[], invalidRows: RowError[], requestId: UUID}`. FieldError = {field: string, reason: string}; RowError = {line: int >= 2 for CSV or >= 1 for USDA index, reason: string, sourceKey: string or null}. Arrays are always present, even when empty. Never return stack traces, tokens, raw SQL, or secrets. Proxy outages may return an HTML gateway error; C1 treats any non-JSON upstream failure as an unavailable state.

I5: food search

Element	Contract
Endpoint / permission	GET `/api/v1/search`; any authenticated active user. No request body.
Required inputs	`type`: query string, exactly MEAL or GROCERY. `targetCalories`: decimal digit string parsed as int, 100..5000; reject decimal points, exponents, signs, empty text, and repeated singleton parameters.
Optional inputs	`restaurantIds`: comma-separated canonical UUIDs, maximum 20; omit for all restaurants; deduplicate; reject empty elements and use only with MEAL. `limit`: integer 1..20, default 20.
Success	200 SearchResponse. `results` sorted totalCalories descending, id ascending, all <= target; maximum limit. Empty matches: results [], closestOption is the lowest active scoped option or null.
Validation / authentication errors	400 VALIDATION_ERROR with fieldErrors; 401 AUTH_REQUIRED for missing/invalid/expired credentials or deleted identity; 503 AUTH_UNAVAILABLE if required identity verification cannot complete.
Dependency errors	503 CATALOG_UNAVAILABLE on connection failure or database query timeout. No automatic catalog retry in the request path. C1 offers an explicit retry.
Timing	D0 allocation: capture <= 150 ms; network <= 750 ms; validation/query/rank <= 800 ms (SQL <= 500 ms); render <= 300 ms; total <= 2 seconds. These are budgets to measure, not demonstrated latency.

```
GET /api/v1/search?type=MEAL&targetCalories=700&limit=20
Authorization: Bearer <access-token>
Accept: application/json
```

For the UC-01 fixture containing 450, 680, and 900 kcal meals, return the 680 then 450 entries, with closestOption null. At a target of 300, return `results: []` and the 450-kcal entry as closestOption. C1 displays "No meals found at or below 300 calories" and labels that 450-kcal alternative as above the target. If the selected restaurant has no active meals, no unrelated restaurant is substituted.

Boundary example: `targetCalories=abc` returns 400 with code VALIDATION_ERROR and a field error stating "Enter a whole number from 100 to 5000." C2 does not query the catalog. The same behavior applies to -50, 99, 5001, and 100.5. The exact valid endpoints 100 and 5000 are accepted.

I7: nutrition import preview and confirmation

Element	Contract
Preview endpoint	POST `/api/v1/imports`; requires maintainer role. C1 validates CSRF for browser-origin state-changing requests.
CSV request	multipart/form-data: restaurantId: required UUID of an existing restaurant; file: required UTF-8 CSV <= 5 MiB; source: required literal CSV. Exactly one source mode. CSV columns and validation are defined in A2.
USDA request	application/json: `{source: "USDA", query: string 1..200, fdcIds: int[]}`; query required, fdcIds optional selection of <= 1000 positive unique IDs. Without fdcIds, first return candidate records for explicit maintainer selection; this 200 CANDIDATES response is not a staged import. Fetch details for selected IDs and normalize before preview. No restaurantId or file in this mode.
Candidate response	`{status: "CANDIDATES", candidates: [{fdcId: int, name: string}], nextPage: int or null}`. `page`: optional positive integer on candidate requests, default 1; page size 100. Maintainer must select IDs before any catalog preview is created.
Preview success	201 ImportResponse for PENDING_CONFIRMATION; 200 ImportResponse for NO_CHANGES. Live foods/meals are untouched. Store preview for 24 hours; it is tied to its creator's subject and maintainer role.
Confirm endpoint	POST `/api/v1/imports/{id}/confirm`; id: required UUID; no body. Creator must still have maintainer role. 200 ImportResponse with APPLIED, including on duplicate confirmation.
View preview	GET `/api/v1/imports/{id}`; creator with maintainer role; returns 200 ImportResponse or 404. This endpoint restores staged work after refresh.
Validation errors	400 INVALID_REQUEST for wrong mode or malformed envelope; 404 RESTAURANT_NOT_FOUND; 413 FILE_TOO_LARGE; 415 UNSUPPORTED_MEDIA_TYPE; 422 IMPORT_INVALID with all invalidRows collected within row/byte limits; reject entire input.
Confirmation errors	404 IMPORT_NOT_FOUND (also conceals other users' batches); 409 CATALOG_CHANGED or INVALID_IMPORT_STATE; 410 IMPORT_EXPIRED. None writes live nutrition data.
Shared / dependency errors	401 AUTH_REQUIRED; 403 MAINTAINER_REQUIRED; 429 SOURCE_RATE_LIMITED after bounded source retry; 502 SOURCE_UNAVAILABLE or SOURCE_INVALID; 503 CATALOG_UNAVAILABLE. All use ErrorResponse.

ImportResponse: importId: UUID or null for NO_CHANGES; status: PENDING_CONFIRMATION/APPLIED/NO_CHANGES/REJECTED/FAILED/EXPIRED; added: int; removed: int; changed: ChangedItem[]; mealsRecalculated: int; expiresAt: ISO UTC string or null; baseCatalogVersion: int64. ChangedItem = {sourceKey: string, itemName: string, oldCalories: int or null, newCalories: int or null, changedFields: string[]}. Name/serving/source changes count as changes even if calories are equal. New/removed counts exclude changed rows. The preview reports 0 recalculated meals; APPLIED reports the actual number.

Import examples, versioning, and internal methods

```
POST /api/v1/imports
Content-Type: multipart/form-data; boundary=upload
Authorization: Bearer <maintainer-access-token>

--upload
Content-Disposition: form-data; name="source"

CSV
--upload
Content-Disposition: form-data; name="restaurantId"

22222222-2222-4222-8222-222222222222
--upload
Content-Disposition: form-data; name="file"; filename="nutrition.csv"
Content-Type: text/csv

itemId,name,calories,servingSize,sourceUrl
burger-1,Burger,550,1 burger,https://example.org/nutrition
--upload--
```

The CSV above is a complete snapshot for its restaurant; omitted existing IDs are explicitly previewed as removed. This example is illustrative and is not meant to be applied to a real restaurant catalog.

```
{
  "importId": "33333333-3333-4333-8333-333333333333",
  "status": "PENDING_CONFIRMATION", "added": 0, "removed": 0,
  "changed": [{"sourceKey": "burger-1", "itemName": "Burger",
    "oldCalories": 500, "newCalories": 550,
    "changedFields": ["calories"]}],
  "mealsRecalculated": 0, "expiresAt": "2026-11-11T00:00:00Z",
  "baseCatalogVersion": 7
}
```

Internal methods: `SearchService.search(SearchQuery, AuthenticatedUser) -> SearchResponse` calls `CatalogRepository.findMatches(query) -> List<ResultRow>` and `loadComponents(UUID[]) -> Map<UUID, List<ItemResult>>`. `ImportService.preview(ImportInput, MaintainerSubject) -> ImportResponse` calls a source adapter, row validator, diff builder, and staging repository. `ImportService.confirm(UUID, MaintainerSubject) -> ImportResponse` owns the version lock and transactional writer. Validation raises typed domain errors mapped once by controller advice.

Versioning: v1 covers paths, parameter meanings, response names/types, calorie units, sorting, empty-result semantics, and error codes. The `quantity`, `roundingAdjustmentCalories`, preview lookup, USDA selection, and named error envelope refine D0's sketches and are logged in D2. Adding optional fields is compatible with clients that ignore unknown fields. Removing/renaming fields, changing units, making optional inputs required, or changing ranking requires v2 or a coordinated published migration. Do not silently change v1 behavior to protein ranking.

6. Technology choices with justification

Database: PostgreSQL with explicit SQL access

Team skill fit: D0 records that Christopher and Brayden have PostgreSQL experience; its relational model also matches the team's SQL coursework. **Licensing:** PostgreSQL uses the PostgreSQL License, permitting use without a database license fee [R1]. **Community support:** Its maintained reference manual and established SQL ecosystem cover constraints, transactions, and index plans. **Performance:** Composite/partial B-tree indexes and precomputed totals support a bounded read; a confirmation transaction updates ingredients, totals, and the batch together. We still must measure the broad restaurant-filter plan. **Cost and hosting:** One local database container with a persistent volume fits the existing VM and avoids an additional hosted database bill. We passed over MongoDB because meal composition and atomic cross-table updates are central, and MySQL because PostgreSQL is already the D0 choice and team familiarity removes a migration cost; neither alternative is claimed incapable.

Backend: Java and Spring Boot modular monolith

Team skill fit: D0 reports existing Spring Boot experience, so keep C2/C3/C4 in separate packages of one backend image. **Licensing:** Spring Boot and Spring Security are Apache-2.0 [R2]. **Community support:** Official Spring documentation covers MVC, JDBC, transactions, validation, and JWT resource servers [R3]. **Performance:** Reuse connection pooling and bounded SQL rather than fetch and sort the whole catalog; limit payload sizes and component loading. **Cost and hosting:** One JVM process and deployment fits the single VM; memory use must be measured before increasing replicas. We passed over Express because it would switch the team's established backend language and require an equivalent validation/transaction setup, and separate microservices because they add deployment and failure-handling work without an independent scaling requirement. Use supported, pinned Java and Spring versions; D0 did not specify exact release numbers.

Frontend: Next.js with TypeScript and semantic HTML

Team skill fit: D0 records React/TypeScript familiarity. **Licensing:** Next.js and React use MIT licenses [R4]. **Community support:** Official Next.js authentication and accessibility guidance supports the server/browser boundary [R5]. **Performance:** Render small result lists and request only the selected search scope; server rendering provides initial semantic HTML, while subsequent search updates are interactive. **Cost and hosting:** The frontend can run as the existing local container and does not require a managed frontend subscription. We passed over a browser-only React SPA because D0 explicitly places access/refresh tokens on the server, and a separate templating stack because it gives up the team's React experience without a project requirement. Keep authentication in a maintained OIDC client rather than writing protocol code.

Identity provider: self-hosted Keycloak

Team skill fit: D0 says Christopher has integrated it before. **Licensing:** Keycloak is Apache-2.0 [R6]. **Community support:** Its official administration, OIDC, and admin REST guidance supports login, roles, and deletion [R7]. **Performance:** Local JWT signature verification handles ordinary requests, with bounded identity checks added for deletion correctness in D2; latency and outage behavior must be tested within the search budget. **Cost and hosting:** A container on the existing VM avoids per-user authentication fees but consumes VM memory and requires configuration/backups. We passed over handwritten passwords because the team would own avoidable credential risks, and a hosted provider because D0 prioritizes a free local stack. E1 owns credentials; C5 stores only the subject and calorie target.

External data and integration

Team skill fit: REST/JSON, CSV, and Java HTTP integration fit the team's stated API/database experience. **Licensing:** USDA data is public-domain data under its API guide; retain its recommended attribution [R8]. Restaurant nutrition sources require checking the relevant source's usage terms; public availability does not automatically authorize every redistribution. **Community support:** USDA publishes an API guide and nutrient-basis documentation; restaurant CSV has a project-owned schema and validator. **Performance:** Import and normalize offline; never call USDA on the two-second user search path. The current USDA default limit is documented as 1,000 requests/hour/IP, so use response rate headers rather than assume the D0 wording "per key" [R8]. **Cost and hosting:** Free public data fits the budget, but the maintainer bears the manual CSV update effort. We passed over a paid nutrition API because D0 sets a free-data constraint, and web scraping because source structure and permitted usage can change. No broker or event bus is needed for bounded, maintainer-triggered batches; HTTP plus durable database staging is the chosen messaging approach.

Hosting: Debian VM, Docker Compose, Cloudflare Tunnel

Team skill fit: This is the exact configured deployment in D0; Christopher's experience supports its setup. **Licensing:** Docker Engine/Compose and cloudflared are Apache-2.0 projects; Debian contains individually licensed packages. The Cloudflare hosted service has separate terms and plan limits [R9]. **Community support:** The projects provide deployment and routing documentation; keep configuration in the repository without secret values. **Performance:** Private-network calls avoid extra interservice network hops; a single host is also a failure domain and must be load-tested. **Cost and hosting:** Budget is the already-planned VM plus any domain/backup charges, not a claim that total hosting is zero. D0 assumes an eligible free tunnel plan; verify that assumption before deployment. We passed over Kubernetes for this three-person, one-host project, and over managed multi-vendor hosting because it spreads operations and potential costs across several services. C5 and Keycloak volumes persist; health checks, restart policies, and backup restoration are completed in D2.

Official reference register

Project requirements, team experience, ownership, and D0 decisions come from the supplied assignment screenshots. The following official references support the technology and protocol choices; they do not introduce additional product requirements. Accessed November 9, 2026. The numerical capacity limits and algorithms in this document are proposed design choices.

Reference	Primary source
R1 PostgreSQL	License and index documentation.
R2 Spring licensing	Spring Boot license and Spring Security license.
R3 Spring behavior	Servlet JWT resource server and transaction management.
R4 Frontend licensing	Next.js license and React license.
R5 Next.js	Authentication and accessibility.
R6 Keycloak license	Keycloak license.
R7 Keycloak behavior	Server administration and admin REST API.
R8 USDA	API guide: request limits, licensing, authentication and data documentation.
R9 Deployment	Docker Compose, Cloudflare Tunnel, cloudflared license, and Debian licensing.
R10 Reuse details	Commons CSV license, Jackson databind license, JDBC driver license, and openid-client.

Submission source files: D1_Detailed_Design.md, D1_Data_Model.drawio, D1_Search_Flow.drawio, and D1_Import_State.drawio. Matching SVG exports are included for convenient viewing. All diagrams are editable in diagrams.net or the VS Code Draw.io integration. Place these beside D0 in the repository's Design_Diagrams folder. This document is a design specification, not evidence that the application has been implemented or benchmarked.